

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра интеллектуальных информационных технологий

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине
«Аппаратное и программное обеспечение сетей»

Тема: «Программирование сетевых приложений
на базе платформы Java и библиотеки java.net»

Вариант 11

Выполнил:
студент 3 курса
группы ИИ-28
Цибульскас В.С.

Проверила:
Прохорова С.С.

Цель работы

Цель работы состоит в том, чтобы изучить классы пакета java.net и собрать на их основе TCP-сервер и TCP-клиент по варианту 11. Сервер должен создавать текстовый файл по команде save. Клиент должен подключаться командой connect и отправлять введённую строку по нажатию клавиши PgUp.

Ход работы

Ниже приведены листинги TcpServer.java и TcpClient.java.

TcpServer.java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class TcpServer {
    public static final int PORT = 8080;
    private static final String STOP_MARK = "#end#";

    public static void main(String[] args) throws IOException {
        ServerSocket gate = new ServerSocket(PORT);
        System.out.println("TCP-сервер запущен, порт " + PORT);
        try {
            while (true) {
                Socket incoming = gate.accept();
                new Thread(new Session(incoming), "client-" + incoming.getPort()).start();
            }
        } finally {
            gate.close();
        }
    }

    static final class Session implements Runnable {
        private final Socket link;

        Session(Socket link) {
            this.link = link;
        }

        @Override
        public void run() {
            String peer = String.valueOf(link.getRemoteSocketAddress());
            System.out.println("Подключение: " + peer);
            try {
                Socket socket = link;
                BufferedReader reader = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));
                PrintWriter writer = new PrintWriter(
                    new BufferedWriter(new OutputStreamWriter(socket.getOutputStream())),
                    true);
            } {
                writer.println("Hello, Student!");
                String line;
                while ((line = reader.readLine()) != null) {
                    if (line.startsWith("save ")) {
                        if (!writeSavedFile(line.substring(5).trim(), reader, writer)) {
                            break;
                        }
                    }
                    else {
                        writer.println(line);
                    }
                }
            }
        }
    }
}
```

```

        }
    }
    catch (IOException error) {
        System.err.println("Ошибка сеанса: " + error.getMessage());
    }
    System.out.println("Сеанс завершён: " + peer);
}

private boolean writeSavedFile(String fileName, BufferedReader reader, PrintWriter writer)
    throws IOException {
    if (!acceptableName(fileName)) {
        writer.println("Ошибка: имя файла недопустимо. Соединение закрывается.");
        return false;
    }

    StringBuilder body = new StringBuilder();
    boolean markerSeen = false;
    String row;
    while ((row = reader.readLine()) != null) {
        int markerAt = row.indexOf(STOP_MARK);
        if (markerAt >= 0) {
            body.append(row, 0, markerAt);
            markerSeen = true;
            break;
        }
        body.append(row).append('\n');
    }
    if (!markerSeen) {
        writer.println("Ошибка: маркер #end## не получен. Соединение закрывается.");
        return false;
    }

    try (FileWriter file = new FileWriter(fileName)) {
        file.write(body.toString());
    } catch (IOException error) {
        writer.println("Ошибка: файл " + fileName + " не создан. Соединение закрывается.");
        return false;
    }

    writer.println("Файл " + fileName + " создан, символов: " + body.length());
    return true;
}

private static boolean acceptableName(String fileName) {
    if (fileName.isEmpty() || fileName.length() > 80) {
        return false;
    }
    if (fileName.equals(".") || fileName.equals("..")) {
        return false;
    }
    for (int i = 0; i < fileName.length(); i++) {
        char symbol = fileName.charAt(i);
        if (symbol < 32 || symbol == '/' || symbol == '\\ || symbol == ':') {
            return false;
        }
    }
    return true;
}
}
}

```

TcpClient.java

```

import java.awt.BorderLayout;
import java.awt.KeyboardFocusManager;
import java.awt.event.KeyEvent;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;

```

```

import java.io.PrintWriter;
import java.net.Socket;
import java.time.Instant;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.logging.FileHandler;
import java.util.logging.Formatter;
import java.util.logging.Level;
import java.util.logging.LogRecord;
import java.util.logging.Logger;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

public class TcpClient extends JFrame {
    private static final Logger JOURNAL = Logger.getLogger("aipos.tcp.client");
    private static final DateTimeFormatter CLOCK = DateTimeFormatter
        .ofPattern("yyyy-MM-dd HH:mm:ss")
        .withZone(ZoneId.systemDefault());

    private Socket socket;
    private PrintWriter outgoing;
    private boolean sessionOpen;

    private final JTextArea screen = new JTextArea();
    private final JTextField field = new JTextField();

    public TcpClient() throws IOException {
        super("TCP-клиент, вариант 11");
        openJournal();

        screen.setEditable(false);
        screen.setLineWrap(true);
        add(new JScrollPane(screen), BorderLayout.CENTER);

        JButton sendButton = new JButton("Отправить");
        sendButton.addActionListener(event -> submit());
        KeyboardFocusManager.getCurrentKeyboardFocusManager().addKeyEventDispatcher(event -> {
            if (event.getID() != KeyEvent.KEY_PRESSED || !isFocused()) {
                return false;
            }
            int code = event.getKeyCode();
            boolean pageUp = code == KeyEvent.VK_PAGE_UP;
            boolean upInField = code == KeyEvent.VK_UP && event.getComponent() == field;
            if (!pageUp && !upInField) {
                return false;
            }
            submit();
            return true;
        });

        JPanel bottom = new JPanel(new BorderLayout(6, 0));
        bottom.add(field, BorderLayout.CENTER);
        bottom.add(sendButton, BorderLayout.EAST);
        add(bottom, BorderLayout.SOUTH);

        setSize(620, 420);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        addWindowListener(new java.awt.event.WindowAdapter() {
            @Override
            public void windowClosing(java.awt.event.WindowEvent event) {
                closeLink();
            }
        });

        say("Подключение: connect <адрес> <порт>");
        say("Отправка строки: PgUp. На MacBook это Fn + стрелка вверх");
        say("Можно также нажать стрелку вверх в поле ввода или кнопку «Отправить»");
    }
}

```

```

        say("Журнал сеанса пишется в файл protocol.log");
        setVisible(true);
        field.requestFocusInWindow();
    }

    private static void openJournal() throws IOException {
        FileHandler handler = new FileHandler("protocol.log", true);
        handler.setFormatter(new Formatter() {
            @Override
            public String format(LogRecord record) {
                String moment = CLOCK.format(Instant.ofEpochMilli(record.getMillis()));
                return moment + " " + record.getMessage() + System.lineSeparator();
            }
        });
        JOURNAL.setUseParentHandlers(false);
        JOURNAL.setLevel(Level.INFO);
        JOURNAL.addHandler(handler);
    }

    private void submit() {
        String text = field.getText();
        field.setText("");
        if (text.isEmpty()) {
            say("Поле пустое: сначала введите строку");
            return;
        }
        if (text.equals("connect") || text.startsWith("connect ")) {
            connectTo(text);
            return;
        }
        synchronized (this) {
            if (outgoing == null) {
                say("Сначала выполните: connect <адрес> <порт>");
                return;
            }
            outgoing.println(text);
        }
        say(">>> " + text);
    }

    private void connectTo(String command) {
        String tail = command.substring("connect".length()).trim().replace(':', ' ');
        String[] parts = tail.isEmpty() ? new String[0] : tail.split("\\s+");
        if (parts.length != 2) {
            say("Формат команды: connect <адрес> <порт>");
            return;
        }
        final String host = parts[0];
        final int port;
        try {
            port = Integer.parseInt(parts[1]);
            if (port < 1 || port > 65535) {
                say("Порт должен быть в диапазоне 1..65535");
                return;
            }
        } catch (NumberFormatException error) {
            say("Порт должен быть числом");
            return;
        }

        synchronized (this) {
            if (outgoing != null) {
                say("Соединение уже установлено");
                return;
            }
            try {
                socket = new Socket(host, port);
                outgoing = new PrintWriter(
                    new BufferedWriter(new OutputStreamWriter(socket.getOutputStream())),
                    true);
                sessionOpen = true;
                JOURNAL.info("CONNECTION START " + host + ":" + port);
            }
        }
    }

```

```

        say("Подключен к " + host + ":" + port);
        Thread reader = new Thread(this::readReplies, "tcp-reader");
        reader.setDaemon(true);
        reader.start();
    } catch (IOException error) {
        say("Подключение не удалось: " + error.getMessage());
        silentClose();
    }
}

private void readReplies() {
    Socket current;
    synchronized (this) {
        current = socket;
    }
    if (current == null) {
        return;
    }
    try {
        BufferedReader incoming = new BufferedReader(
            new InputStreamReader(current.getInputStream()));
        String line;
        while ((line = incoming.readLine()) != null) {
            final String received = line;
            JOURNAL.info("RECV " + received);
            SwingUtilities.invokeLater(() -> say("<<< " + received));
        }
        SwingUtilities.invokeLater(() -> say("Соединение закрыто"));
    } catch (IOException error) {
        SwingUtilities.invokeLater(() -> say("Соединение закрыто"));
    }
    closeLink();
}

private synchronized void closeLink() {
    if (!sessionOpen) {
        return;
    }
    sessionOpen = false;
    JOURNAL.info("CONNECTION END");
    silentClose();
}

private void silentClose() {
    if (socket != null) {
        try {
            socket.close();
        } catch (IOException ignored) {
        }
    }
    socket = null;
    outgoing = null;
}

private void say(String message) {
    screen.append(message + "\n");
    screen.setCaretPosition(screen.getDocument().getLength());
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        try {
            new TcpClient();
        } catch (IOException error) {
            error.printStackTrace();
        }
    });
}
}

```

Вывод

Собран многопоточный TCP-сервер на классах `ServerSocket` и `Socket`. Команда `save` отделяет имя файла и записывает тело через `FileWriter` до маркера `#end##`. Недопустимое имя или сбой создания файла приводят к уведомлению клиента и закрытию только его сеанса. Клиент на `Swing` открывает соединение командой `connect`, отправляет введенную строку клавишей `PgUp` и ведёт журнал начала, окончания и принятых ответов. Проверка на локальном порту 8080 подтвердила приветствие, эхо, запись файла и разрыв соединения при ошибке.